



**Assignment 1 — Python Fundamentals for Data Streaming
AITU Campus Shuttle and Mobility Data**

Student full name: Ayazhan Sailauova

Student ID: 240791

Group: BDA-2402

Instructor: Azamat Serek

Submission date: 14.09.2026

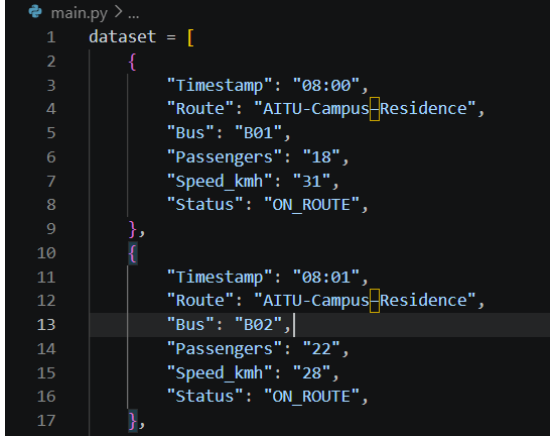
Git repository: https://github.com/avayashka/RTDA_Assignment1.git

Python version: Python 3.14

1. Work Record

Item	Student entry
Work start date	10.09.2026
Work completion date	14.09.2026
Main tools / libraries used	Python, FastAPI, JSON, datetime, uvicorn
Source-code file names	main.py, api.py
Test file / notebook name	tests.py
README location	README.md

2.1. Data Structures and Parsing

Required evidence	Student response / reference
Explanation	Based on a synthetic input data, I created a dataset as a list of dictionaries of the “key”: “value” type in the “main.py” file. I installed the datetime module to ensure the correct datatype of the “Timestamp” feature. This module helps to convert the input strings into a datetime object. For numeric variables, I set the validation rule so that the value should be non-negative. Those columns with numeric datatype were parsed using the int() and float() functions. The validate_events(event) function accepts the event and validates it by executing each try-except command that verifies the correctness of the input data about the event. Then, by using the events_stream generator, the events are accessed one at a time.
Code / file / function reference	main.py: - The list of dictionaries  - Timestamps validation <pre>timestamp = datetime.strptime(event["Timestamp"], "%H:%M")</pre> - Numeric variables validation

```

try:
    passengers = int(event.get("Passengers"))
    if passengers < 0:
        errors.append("Passengers count must be a non-negative integer")
except (ValueError, TypeError):
    passengers = None
    errors.append("Invalid Passengers value")

try:
    speed = float(event.get("Speed_kmh"))
    if not (0 <= speed <= 120):
        errors.append("Speed of a bus is out of bounds")
except (ValueError, TypeError):
    speed = None
    errors.append("Invalid Speed value")

```

4. The event validation function:

```

def validate_events(event):
    errors = []
    try:
        timestamp = datetime.strptime(event["Timestamp"], "%H:%M")
        formatted_timestamp = timestamp.strftime("%H:%M")
    except (KeyError, TypeError, ValueError):
        formatted_timestamp = None
        errors.append("Invalid timestamp format")

    valid_routes = ["AITU-Campus-Residence", "AITU-Campus-Residence"]
    if event.get("Route") not in valid_routes:
        errors.append("Invalid route specified")

    try:
        timestamp = datetime.strptime(event["Timestamp"], "%H:%M")
        formatted_timestamp = timestamp.strftime("%H:%M")
    except (KeyError, TypeError, ValueError):
        formatted_timestamp = None
        errors.append("Invalid timestamp format")

    try:
        passengers = int(event.get("Passengers"))
        if passengers < 0:
            errors.append("Passengers count must be a non-negative integer")
    except (ValueError, TypeError):
        passengers = None
        errors.append("Invalid Passengers value")

    try:
        speed = float(event.get("Speed_kmh"))
        if not (0 <= speed <= 120):
            errors.append("Speed of a bus is out of bounds")
    except (ValueError, TypeError):
        speed = None
        errors.append("Invalid Speed value")

    status = str(event.get("Status")).upper()
    if status not in ["ON_ROUTE", "STOPPED"]:
        errors.append("Invalid Status")

    if errors:
        return {"accepted": False, "errors": errors}

    return {
        "accepted": True,
        "event": {
            **event,
            "Timestamp": formatted_timestamp,
            "Passengers": passengers,
            "Speed_kmh": speed,
            "Status": status
        },
        "errors": [],
    }

```

	5. A generator: <pre>def events_stream(events_list): for event in events_list: validation = validate_events(event) if validation["accepted"]: yield event</pre>
Execution evidence / result	As a result, each input value of a validated event was converted to a required datatype and stored in a list of dictionaries. Later, this dataset was uploaded to JSON file.
Student-specific observation or decision	The input dataset was written directly to “main.py” file. Using an event validator, the correct datatype was assigned to each value. This method helps to prevent the invalid datatypes and not accessible events.

2.2. Event Validation

Required evidence	Student response / reference
Explanation	For each attribute of a dataset, there is a validation rule. For example, the value of a “Timestamp” must be of %H:%M format. The number of passengers is always non-negative value, so that <code>int(event.get("Passengers"))</code> must be more or equal to 0. There are boundaries of an accepted speed of a bus from 0 to 120 km per hour. Therefore, if a bus speed is out of [0; 120] range, then there will be an error of invalid value. For a status variable, this key only accepts two values: “ON_ROUTE” or “STOPPED”. If those validations were not accepted, then the event was considered as invalid. For the route of bus, I created the validation rule so that it accepts only the busses with “AITU-Campus–Residence” route.
Code / file / function reference	<pre>def validate_events(event): errors = [] try: timestamp = datetime.strptime(event["Timestamp"], "%H:%M") formatted_timestamp = timestamp.strftime("%H:%M") except (KeyError, TypeError, ValueError): formatted_timestamp = None errors.append("Invalid timestamp format") valid_routes = ["AITU-Campus–Residence", "AITU-Campus-Residence"] if event.get("Route") not in valid_routes: errors.append("Invalid route specified") try: timestamp = datetime.strptime(event["Timestamp"], "%H:%M") formatted_timestamp = timestamp.strftime("%H:%M") except (KeyError, TypeError, ValueError): formatted_timestamp = None errors.append("Invalid timestamp format")</pre>

	<pre> try: passengers = int(event.get("Passengers")) if passengers < 0: errors.append("Passengers count must be a non-negative integer") except (ValueError, TypeError): passengers = None errors.append("Invalid Passengers value") try: speed = float(event.get("Speed_kmh")) if not (0 <= speed <= 120): errors.append("Speed of a bus is out of bounds") except (ValueError, TypeError): speed = None errors.append("Invalid Speed value") status = str(event.get("Status")).upper() if status not in ["ON_ROUTE", "STOPPED"]: errors.append("Invalid Status") if errors: return {"accepted": False, "errors": errors} return { "accepted": True, "event": { **event, "Timestamp": formatted_timestamp, "Passengers": passengers, "Speed_kmh": speed, "Status": status }, "errors": [], } </pre>
Execution evidence result	Based on validation rules, the event: <pre> { "Timestamp": "08:05", "Route": "AITU-Campus-Residence", "Bus": "B02", "Passengers": "-3", "Speed_kmh": "30", "Status": "ON_ROUTE", } </pre> will not be accepted because it contains the negative integer value -7. Another event: <pre> { "Timestamp": "08:05", "Route": "AITU-Campus-Residence", </pre>

	<pre> "Bus": "B02", "Passengers": "26", "Speed_kmh": "135", "Status": "ON_ROUTE", }, </pre> will output an error of the speed value out of allowed range. The event <pre> { "Timestamp": "08:10", "Route": "EXPO-AITU", "Bus": "B01", "Passengers": "16", "Speed_kmh": "30", "Status": "ON_ROUTE", }, </pre> Will also not accepted because of different route. The perfect example of an event that will be accepted is this one: <pre> { "Timestamp": "08:10", "Route": "AITU-Campus-Residence", "Bus": "B01", "Passengers": "16", "Speed_kmh": "30", "Status": "ON_ROUTE", }, </pre>
Student-specific observation or decision	The validation rules in “main.py” file helps to stream the data process correctly.

2.3. Streaming Generator

Required evidence	Student response / reference
Explanation	The events_stream generator produces sequential events that were validated at the previous step. The chronological order of events is verified by comparing the value of the “Timestamp” variable with the next event’s value.
Code / file / function reference	main.py:

	<pre>def events_stream(events_list): for event in events_list: validation = validate_events(event) if validation["accepted"]: yield event def events_chronology(stream): last = None for event in stream: current = event.get("Timestamp") if last is not None: if current < last: print("Chronology Error") last = current print(event)</pre> <pre>stream = events_stream(dataset) events_chronology(stream)</pre>
Execution evidence / result	<pre>PS C:\Users\MB\Desktop\RTDA\Assignment1> py main.py {'Timestamp': '08:00', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '18', 'Speed_kmh': '31', 'Status': 'ON_ROUTE'} {'Timestamp': '08:01', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '22', 'Speed_kmh': '28', 'Status': 'ON_ROUTE'} {'Timestamp': '08:02', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '21', 'Speed_kmh': '29', 'Status': 'ON_ROUTE'} {'Timestamp': '08:03', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '25', 'Speed_kmh': '27', 'Status': 'ON_ROUTE'} {'Timestamp': '08:04', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '24', 'Speed_kmh': '0', 'Status': 'STOPPED'} {'Timestamp': '08:05', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '26', 'Speed_kmh': '30', 'Status': 'ON_ROUTE'} {'Timestamp': '08:06', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '23', 'Speed_kmh': '32', 'Status': 'ON_ROUTE'} {'Timestamp': '08:07', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '28', 'Speed_kmh': '26', 'Status': 'ON_ROUTE'} {'Timestamp': '08:08', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '27', 'Speed_kmh': '25', 'Status': 'ON_ROUTE'} {'Timestamp': '08:09', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '30', 'Speed_kmh': '24', 'Status': 'ON_ROUTE'}</pre>
Student-specific observation or decision	The use of generator is preferred when we work with large data that is continuous. This makes the stream process more memory efficient. The yield keyword helps to produce a value without ending the function. As a result, all 10 events were processed in a chronological order.

2.4. Streaming Simulation

Required evidence	Student response / reference
Explanation	When the mai.py file is executed, the stream simulation starts. At the first step, the events pass through the set of validation rules to set the correct datatypes and accepted values of events. Then, the processor yields validated events one at a time by creating the chronological sequence. This consumes less memory allocation. The collected dataset is then parsed to json file.
Code / file / function reference	Main.py: <pre>from datetime import datetime def validate_events(event): errors = [] try: timestamp = datetime.strptime(event["Timestamp"], "%H:%M") formatted_timestamp = timestamp.strftime("%H:%M") except (KeyError, TypeError, ValueError): formatted_timestamp = None</pre>

```

        errors.append("Invalid timestamp format")

    try:
        passengers = int(event.get("Passengers"))
        if passengers < 0:
            errors.append("Passengers count must be a non-negative integer")
    except (ValueError, TypeError):
        passengers = None
        errors.append("Invalid Passengers value")

    try:
        speed = float(event.get("Speed_kmh"))
        if not (0 <= speed <= 120):
            errors.append("Speed of a bus is out of bounds")
    except (ValueError, TypeError):
        speed = None
        errors.append("Invalid Speed value")

    status = str(event.get("Status")).upper()
    if status not in ["ON_ROUTE", "STOPPED"]:
        errors.append("Invalid Status")

    if errors:
        return {"accepted": False, "errors": errors}

    return {
        "accepted": True,
        "event": {
            **event,
            "Timestamp": formatted_timestamp,
            "Passengers": passengers,
            "Speed_kmh": speed,
            "Status": status
        },
        "errors": [],
    }

def events_stream(events_list):
    for event in events_list:
        validation = validate_events(event)
        if validation["accepted"]:
            yield event

def events_chronology(stream):
    last = None
    for event in stream:
        current = event.get("Timestamp")
        if last is not None:

```

	<pre> if current < last: print("Chronology Error") last = current print(event) if __name__ == "__main__": stream = events_stream(dataset) events_chronology(stream) </pre>
Execution evidence / result	<pre> PS C:\Users\MB\Desktop\RTDA\Assignment1> py main.py {'Timestamp': '08:00', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '18', 'Speed_kmh': '31', 'Status': 'ON_ROUTE'} {'Timestamp': '08:01', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '22', 'Speed_kmh': '28', 'Status': 'ON_ROUTE'} {'Timestamp': '08:02', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '21', 'Speed_kmh': '29', 'Status': 'ON_ROUTE'} {'Timestamp': '08:03', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '25', 'Speed_kmh': '27', 'Status': 'ON_ROUTE'} {'Timestamp': '08:04', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '24', 'Speed_kmh': '0', 'Status': 'STOPPED'} {'Timestamp': '08:05', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '26', 'Speed_kmh': '30', 'Status': 'ON_ROUTE'} {'Timestamp': '08:06', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '23', 'Speed_kmh': '32', 'Status': 'ON_ROUTE'} {'Timestamp': '08:07', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '28', 'Speed_kmh': '26', 'Status': 'ON_ROUTE'} {'Timestamp': '08:08', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '27', 'Speed_kmh': '25', 'Status': 'ON_ROUTE'} {'Timestamp': '08:09', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '30', 'Speed_kmh': '24', 'Status': 'ON_ROUTE'} </pre>
Student-specific observation or decision	The simulation of a stream validates accepted events and add them to a processed dataset. The invalid events are excluded from the input data. <pre> if __name__ == "__main__": stream = events_stream(dataset) events_chronology(stream) </pre> - This code line initialize the stream generator when the “main.py” file is run directly.

2.5. Json and FastApi Endpoint

Required evidence	Student response / reference
Explanation	To convert the dataset into json file, I first imported the json module. Using the json.dump() function, the validated events from the python file are uploaded to json. To implement the FastAPI POST/events, import the FastApi module in “api.py” and validated_events from “main.py” file. The endpoint receives the list of dictionaries with event data, runs the validation step, computes the occupation category based on the number of passengers and returns a response. The occupation category is defined using logical operators: <pre> def occupancy(passengers: int) -> str: if 0 <= passengers <= 10: return "LOW" if 11 <= passengers <= 20: return "MEDIUM" if 21 <= passengers <= 30: return "HIGH" return "OVER_CAPACITY" </pre>
Code / file / function reference	<pre> main.py: import json if __name__ == "__main__": stream = events_stream(dataset) events_chronology(stream) with open("events.json", "w") as file: json.dump(dataset, file) </pre>

	api.py: <pre>@app.post("/events") def receive_event(event: dict): result = validate_events(event) if not result["accepted"]: return { "accepted": False, "errors": result["errors"], "occupancy_category": None } raw_event = result["event"] passengers_count = raw_event["Passengers"] category = occupancy(passengers_count) formatted_event = { "timestamp": raw_event.get("Timestamp"), "route": raw_event.get("Route"), "bus": raw_event.get("Bus"), "passengers": raw_event.get("Passengers"), "speed_kmh": raw_event.get("Speed_kmh"), "status": raw_event.get("Status") } return { "accepted": True, "errors": [], "occupancy_category": category, "event": formatted_event }</pre>
Execution evidence / result	Code Details 200 Response body <pre>{ "accepted": false, "errors": ["Passengers count must be a non-negative integer"], "occupancy_category": null }</pre> Response headers <pre>content-length: 105 content-type: application/json date: Sun, 13 Sep 2020 18:56:33 GMT server: uvicorn</pre> Responses - the event was not validated as it contains a negative passengers count

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/events' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "Timestamp": "08:09",
    "Route": "AITU-Campus-Residence",
    "Bus": "B02",
    "Passengers": "10",
    "Speed_kmh": "145",
    "Status": "ON_ROUTE"
  }'
```

Request URL

http://127.0.0.1:8000/events

Server response

Code	Details
------	---------

200

Response body

```
{
  "accepted": false,
  "errors": [
    "Speed of a bus is out of bounds"
  ],
  "occupancy_category": null
}
```

Response headers

```
content-length: 89
content-type: application/json
date: Sun, 13 Sep 2026 18:57:20 GMT
server: uvicorn
```

Responses

Code	Description
------	-------------

200

Successful Response

- the event was not accepted because the speed of a bus was out of allowed range.

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/events' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "Timestamp": "08:15",
    "Route": "EXPO-Direct",
    "Bus": "B01",
    "Passengers": "15",
    "Speed_kmh": "30",
    "Status": "ON_ROUTE"
  }'
```

Request URL

```
http://127.0.0.1:8000/events
```

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "accepted": false,
  "errors": [
    "Invalid route specified"
  ],
  "occupancy_category": null
}
```

- the event was not accepted because of the different route

	Curl <pre>curl -X 'POST' \ 'http://127.0.0.1:8000/events' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "Timestamp": "08:15", "Route": "AITU-Campus-Residence", "Bus": "B01", "Passengers": "15", "Speed_kmh": "30", "Status": "ON_ROUTE" }'</pre> Request URL <pre>http://127.0.0.1:8000/events</pre> Server response <table border="1"> <thead> <tr> <th>Code</th><th>Details</th></tr> </thead> <tbody> <tr> <td>200</td><td> Response body <pre>{ "accepted": true, "errors": [], "occupancy_category": "MEDIUM", "event": { "timestamp": "08:15", "route": "AITU-Campus-Residence", "bus": "B01", "passengers": 15, "speed_kmh": 30, "status": "ON_ROUTE" } }</pre> </td></tr> </tbody> </table> - the test was executed successfully and event was accepted with medium occupancy	Code	Details	200	Response body <pre>{ "accepted": true, "errors": [], "occupancy_category": "MEDIUM", "event": { "timestamp": "08:15", "route": "AITU-Campus-Residence", "bus": "B01", "passengers": 15, "speed_kmh": 30, "status": "ON_ROUTE" } }</pre>
Code	Details				
200	Response body <pre>{ "accepted": true, "errors": [], "occupancy_category": "MEDIUM", "event": { "timestamp": "08:15", "route": "AITU-Campus-Residence", "bus": "B01", "passengers": 15, "speed_kmh": 30, "status": "ON_ROUTE" } }</pre>				
Student-specific observation or decision	The api.py file ensures the web service to execute the POST/event endpoint and validate the new events by adding information about the occupancy category, accepted status and return the structured json response.				

2.6. Data Analysis

Required evidence	Student response / reference
Explanation	The average passengers count is calculated by sum of passenger among all events and dividing by the number of events. The len(passengers) calculates the number of events with passengers, the sum(passengers) calculates the passengers count of all events. The maximum passengers value is computed using the max(passengers) command. The sum(event["Status"] == "STOPPED" for event in dataset) access the events which status is stopped and

	outputs the number of those events. The logic behind the busiest event is the event with highest passengers count. Using the key value, the program finds the event with maximum passengers value and retrieves the minute and bus number.
Code / file / function reference	<pre> main.py: passengers = [int(event["Passengers"]) for event in dataset] avg_passengers = sum(passengers) / len(passengers) max_passengers = max(passengers) stopped = sum(event["Status"] == "STOPPED" for event in dataset) busiest_event = max(dataset, key=lambda event: int(event["Passengers"])) busiest_minute = busiest_event["Timestamp"] busiest_bus = busiest_event["Bus"] print("The average passengers count = ", avg_passengers) print("The maximum passengers count = ", max_passengers) print("The number of stopped events = ", stopped) print(f"The busiest event was {busiest_bus} Bus at {busiest_minute}") </pre>
Execution evidence / result	<pre> PS C:\Users\NB\Desktop\RTDA\Assignment1> py main.py {'Timestamp': '08:00', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '18', 'Speed_kmh': '31', 'Status': 'ON_ROUTE'} {'Timestamp': '08:01', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '22', 'Speed_kmh': '28', 'Status': 'ON_ROUTE'} {'Timestamp': '08:02', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '21', 'Speed_kmh': '29', 'Status': 'ON_ROUTE'} {'Timestamp': '08:03', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '25', 'Speed_kmh': '27', 'Status': 'ON_ROUTE'} {'Timestamp': '08:04', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '24', 'Speed_kmh': '0', 'Status': 'STOPPED'} {'Timestamp': '08:05', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '26', 'Speed_kmh': '30', 'Status': 'ON_ROUTE'} {'Timestamp': '08:06', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '23', 'Speed_kmh': '32', 'Status': 'ON_ROUTE'} {'Timestamp': '08:07', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '28', 'Speed_kmh': '26', 'Status': 'ON_ROUTE'} {'Timestamp': '08:08', 'Route': 'AITU-Campus-Residence', 'Bus': 'B01', 'Passengers': '27', 'Speed_kmh': '25', 'Status': 'ON_ROUTE'} {'Timestamp': '08:09', 'Route': 'AITU-Campus-Residence', 'Bus': 'B02', 'Passengers': '30', 'Speed_kmh': '24', 'Status': 'ON_ROUTE'} The average passengers count = 24.4 The maximum passengers count = 30 The number of stopped events = 1 The busiest event was B02 Bus at 08:09 </pre>
Student-specific observation or decision	When the “main.py” file is executed, it outputs the set of validates events and the calculated summary statistics.

2.7. Testing and Code Quality

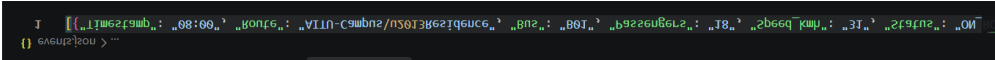
Required evidence	Student response / reference
Explanation	The code test 4 cases of events: test_validate_events(), test_negative_passengers(), test_speed_boundaries(), test_occupancy_category(). For the test_validate_events() the input data is: {"Timestamp": "08:10", "Route": "AITU-Campus-Residence", "Bus": "B01", "Passengers": "20", "Speed_kmh": "31", "Status": "ON_ROUTE"} The expected output is { accepted: True, Passengers: 20, Speed_kmh: 31.0 } . The actual status is PASSED, all data is correct. For the test_negative_passengers() the input data is: {"Timestamp": "08:11", "Route": "AITU-Campus-Residence",

	<pre> "Bus": "B02", "Passengers": "-3", "Speed_kmh": "30", "Status": "ON_ROUTE"} </pre> The expected output is { accepted: False, Error: Passengers count must be a non-negative integer } . The actual status is PASSED, the expected error occurred. For the test_speed_boundaries() the input data is: {"Timestamp": "08:12", "Route": " AITU-Campus-Residence ", "Bus": "B01", "Passengers": "10", "Speed_kmh": "138", "Status": "ON_ROUTE"} The expected output is { accepted: False, Error: Speed of a bus is out of bounds } . The actual status is PASSED, the expected error occurred. For the test_occupancy_category () the input data is: 7, 14, 21 and 42. The expected output is “LOW”, “MEDIUM”, “HIGH”, “OVER_CAPACITY”. The actual status is PASSED, the exact category of occupancy was defined. For the additional test of the route of buses, the input data is: { "Timestamp": "08:13", "Route": "EXPO-Direct", "Bus": "B01", "Passengers": "15", "Speed_kmh": "25", "Status": "ON_ROUTE" }. The expected output is PASSED, with an error of “Invalid route specified” error. The actual status is PASSED with the expected error.
Code / file / function reference	<pre> test.py: import pytest from main import validate_events from api import occupancy def test_validate_events(): sample = {"Timestamp": "08:10", "Route": "AITU-Campus-Residence", "Bus": "B01", "Passengers": "20", "Speed_kmh": "31", "Status": "ON_ROUTE"} result = validate_events(sample) assert result["accepted"] is True assert result["event"]["Passengers"] == 20 </pre>

	<pre> assert result["event"]["Speed_kmh"] == 31.0 def test_negative_passengers(): sample = {"Timestamp": "08:11", "Route": "AITU-Campus-Residence", "Bus": "B02", "Passengers": "-3", "Speed_kmh": "30", "Status": "ON_ROUTE"} result = validate_events(sample) assert result["accepted"] is False assert "Passengers count must be a non-negative integer" in result["errors"] def test_speed_boundaries(): sample = {"Timestamp": "08:12", "Route": "AITU-Campus-Residence", "Bus": "B01", "Passengers": "10", "Speed_kmh": "138", "Status": "ON_ROUTE"} result = validate_events(sample) assert result["accepted"] is False assert "Speed of a bus is out of bounds" in result["errors"] def test_occupancy_category(): assert occupancy(7) == "LOW" assert occupancy(14) == "MEDIUM" assert occupancy(21) == "HIGH" assert occupancy(42) == "OVER_CAPACITY" def test_invalid_route(): sample = { "Timestamp": "08:13", "Route": "EXPO-Direct", "Bus": "B01", "Passengers": "15", "Speed_kmh": "25", "Status": "ON_ROUTE" } result = validate_events(sample) assert result["accepted"] is False assert "Invalid route specified" in result["errors"] </pre>
Execution evidence / result	

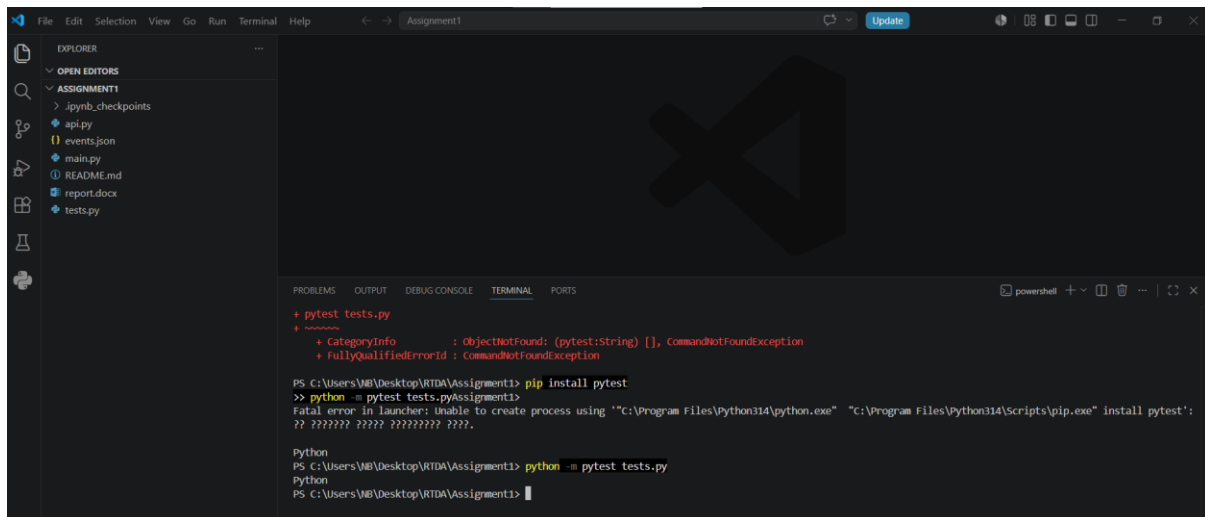
	<pre> PS C:\Users\NB\Desktop\RTDA\Assignment1> py -m pytest tests.py -v ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\NB\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\Users\NB\Desktop\RTDA\Assignment1 plugins: anyio-4.15.1 collected 5 items tests.py::test_validate_events PASSED tests.py::test_negative_passengers PASSED tests.py::test_speed_boundaries PASSED tests.py::test_occupancy_category PASSED tests.py::test_invalid_route PASSED ===== 5 passed in 0.28s ===== </pre>
Student-specific observation or decision	The separate tests.py file helps to maintain the testing cases beyond all python pipelines and API routes.

2.8. Technical Interpretation

Required evidence	Student response / reference
Explanation	Generators provide the faster way to work with large amounts of data. Instead of allocating the memory for a set of data at the initial step, generators yield one object at a time. After the execution, the function stops and returns the value. The generators implementation is preferable when we work with continuously arriving data that require a memory efficient solution. The limitation of the generators is the sequential behavior of streaming data. We can continue the sequence response from the point it stopped, however, we cannot restart it or access the values randomly.
Code / file / function reference	main.py: <pre> def events_stream(events_list): for event in events_list: validation = validate_events(event) if validation["accepted"]: yield event def events_chronology(stream): last = None for event in stream: current = event.get("Timestamp") if last is not None: if current < last: print("Chronology Error") last = current print(event) </pre>
Execution evidence / result	 <pre> [{"Timestamp": "2023-01-01T00:00:00", "Event": "Vehicle entered zone", "Location": "Zone A", "Status": "Active"}, {"Timestamp": "2023-01-01T00:01:00", "Event": "Vehicle exited zone", "Location": "Zone A", "Status": "Completed"}, {"Timestamp": "2023-01-01T00:02:00", "Event": "Vehicle entered zone", "Location": "Zone B", "Status": "Active"}, {"Timestamp": "2023-01-01T00:03:00", "Event": "Vehicle exited zone", "Location": "Zone B", "Status": "Completed"}, {"Timestamp": "2023-01-01T00:04:00", "Event": "Vehicle entered zone", "Location": "Zone A", "Status": "Active"}, {"Timestamp": "2023-01-01T00:05:00", "Event": "Vehicle exited zone", "Location": "Zone A", "Status": "Completed"}] </pre>
Student-specific observation or decision	The implementation of the generators optimized the memory consumption to stream data.

3. Individual Work and Anti-Plagiarism Record

During the code execution, I faced problems with modules and libraries importation. When I tried to run `pytest test.py` first, the problem with “`ObjectNotFound`”. To solve it, I installed the `pytest` using “`pip install pytest`”. After the `pytest tests.py`, `main.py` files were run successfully.



As the one option to improve the work, I can create and run additional event test to validate the route of the bus. The test will ensure that the route of the bus is invalid and reject it. This event case is additional test to the required 3 tests.

```
def test_invalid_route():
    sample = {
        "Timestamp": "08:13",
        "Route": "EXP0-Direct",
        "Bus": "B01",
        "Passengers": "15",
        "Speed_kmh": "25",
        "Status": "ON_ROUTE"
    }
    result = validate_events(sample)
    assert result["accepted"] is False
    assert "Invalid route specified" in result["errors"]
```

The limitation of the project is the way the json file is used. The `json.dump()` function overwrites the json file. If the work requires continuous file operations, it may result of input/output error.

4. Student Declaration of Originality

I confirm that this submission represents my own work. I have not copied another student's source code, calculations, tables, screenshots, test results, API responses, or written explanations. Any external material or reused code has been identified. I understand that the

instructor may request a live demonstration, code modification, or explanation of any submitted component.